

Pruebas Unitarias

JUnit, tests parametrizados y dobles de prueba

1. Qué son las pruebas unitarias?

Una prueba unitaria verifica el correcto funcionamiento de un módulo individual del código —normalmente un método o una clase— de forma aislada, antes de comprobar como se integra con el resto.

Para que una prueba unitaria cumpla su función debe ser:

Característica	Qué significa en la práctica
Automatizable	Se ejecuta sin intervención manual
Completa	Cubre la mayor cantidad posible de código, no solo el camino feliz
Reutilizable	No sirve solo para una ejecución puntual; se puede volver a correr cuando se quiera
Independiente	El resultado no debe depender de que se haya ejecutado otra prueba antes
Profesional	Se documenta y se cuida con la misma calidad que el código de produccion

1. Ventajas de las pruebas unitarias

Las pruebas unitarias traen ventajas claras a medida que el proyecto crece:

- **Confianza al modificar**

Cualquier cambio que rompa algo se detecta enseguida.

- **Seguridad antes de integrar**

Se sabe que cada pieza funciona antes de combinarlas.

- **Documentación viva**

Los tests muestran como se espera que se use cada unidad de código.

- **Separación de interfaz e implementación**

Puedes cambiar como funciona algo por dentro sin tocar los tests si el comportamiento externo no cambia.

- **Localización de errores**

Se depura la unidad problemática en aislamiento, sin revisar todo el sistema a la vez.

2. JUnit — Que es y cómo configurarlo

JUnit es el framework de pruebas unitarias más usado en Java. Permite comprobar que cada unidad de código se comporta como se espera. Se integra con Maven o Gradle para automatizar su ejecución dentro de la integración continua.

Conceptos clave antes de escribir el primer test:

Concepto	Qué es
Prueba unitaria	Verifica una funcionalidad concreta de forma aislada
Assert	Instrucción que comprueba si el comportamiento obtenido coincide con el esperado
Anotación	Marca el papel que juega cada método dentro del ciclo de vida de la prueba
Clase de prueba	Agrupar los métodos de test relacionados con una misma clase de producción

JUnit se añade al proyecto Maven como dependencia de tipo 'test' en el pom.xml:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>
```

2. Escribir el primer test en JUnit 5

Por convención, la clase de test se llama igual que la clase bajo prueba con el sufijo Test. Si la clase es Calculadora, el archivo de test es CalculadoraTest. Se coloca en src/test/java, en el mismo paquete.

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;
class CalculadoraTest {
    Calculadora calc;
    @BeforeEach
    void setUp() {
        calc = new Calculadora();
    }
    @Test
    void sumar_dosPositivos_devuelveResultadoCorrecto() {
        assertEquals(8, calc.sumar(3, 5));
    }
}
```

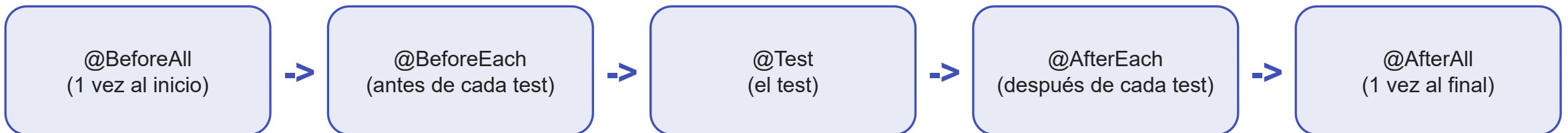
Convención nombre de test: método_situación_resultadoEsperado — legible como documentación sin abrir el código.

2. JUnit — Ciclo de vida de las anotaciones

JUnit ejecuta los métodos de preparacion y limpieza en un orden fijo alrededor de cada `@Test`:

Anotación	Cuando se ejecuta
<code>@BeforeAll</code>	Una sola vez, al principio de toda la clase de pruebas (el método debe ser estatico)
<code>@BeforeEach</code>	Antes de cada método <code>@Test</code> ; se usa para inicializar el estado del test
<code>@Test</code>	El propio método de prueba; JUnit lo ejecuta y evalua sus asserts
<code>@AfterEach</code>	Después de cada método <code>@Test</code> ; se usa para liberar recursos
<code>@AfterAll</code>	Una sola vez, al final de toda la clase de pruebas (el método debe ser estatico)

Orden de ejecución en cada suite:



2. JUnit — Asserts principales

Un assert comprueba que el resultado obtenido coincide con el esperado. Si no coincide, JUnit marca el test como FAILED e informa del valor esperado y el real.

Método	Comprueba que...
<code>assertEquals(esperado, real)</code>	ambos valores son iguales
<code>assertTrue(condición)</code>	la condición es verdadera
<code>assertFalse(condición)</code>	la condición es falsa
<code>assertNull(objeto)</code>	el objeto es null
<code>assertNotNull(objeto)</code>	el objeto no es null
<code>assertThrows(Tipo.class, () -> {...})</code>	el lambda lanza esa excepcion concreta

Ejemplo: `assertThrows` verifica que un método lanza la excepcion esperada:

```
// El test pasa si calc.dividir(10, 0) lanza ArithmeticException
assertThrows(ArithmeticException.class, () -> calc.dividir(10, 0));
```

3. Tests parametrizados

Cuando se quiere probar el mismo método con varios conjuntos de datos distintos, los tests parametrizados ejecutan un único método de prueba varias veces, una por cada conjunto de datos.

La anotación `@ParameterizedTest` se combina con una fuente de datos:

Fuente	Cuando usarla
<code>@ValueSource</code>	Una lista de valores de un mismo tipo
<code>@CsvSource</code>	Varias columnas de datos separadas por comas, directamente en el código
<code>@CsvFileSource</code>	Los datos se leen desde un archivo CSV externo
<code>@MethodSource</code>	Un método aparte genera los datos, útil cuando son más complejos

Ejemplo con `@CsvSource` (cada fila se ejecuta como una llamada independiente):

```
@ParameterizedTest
@CsvSource({
    "2, 3, 5",
    "3, 4, 7"
})
void testSumarCasosVálidos(int a, int b, int esperado) {
    assertEquals(esperado, calculadora.sumar(a, b));
}
```

4. Dobles de prueba — Mocks, Stubs y Dummies

Hay casos en los que probar una clase implica, sin querer, probar también sus dependencias: una base de datos, un servicio web, un archivo externo... Los dobles de prueba son objetos que sustituyen esas dependencias reales dentro del test.

Tipos de dobles

Tipo	Qué hace
Mock	Objeto 'falso' que simula el comportamiento de una dependencia real. Se define que debe devolver cada método y se puede verificar si se llamo y con que parametros.
Stub	Version simplificada de una dependencia que se limita a devolver datos predefinidos, sin la lógica de verificación de un mock.
Dummy	Objeto sin funcionalidad real, que solo existe porque el código lo necesita para ejecutarse (un parametro obligatorio que no influye en el resultado).

Metáfora: Si quieres evaluar las habilidades de un cocinero (la clase bajo prueba) que usa ciertas herramientas (la dependencia), le das herramientas simuladas que sabes que funcionan perfectamente, y así te concentras en evaluar solo al cocinero.

4. Mocks con Mockito

Mockito es la librería más usada en Java para crear mocks. Se añade como dependencia de tipo test en el pom.xml.

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>5.15.2</version>
  <scope>test</scope>
</dependency>
```

Patrón típico de un test con mock:

```
// 1. Crear el mock de la dependencia
Calculadora calc = mock(Calculadora.class);

// 2. Configurar que devuelve cada método
when(calc.sumar(3, 5)).thenReturn(8);
when(calc.multiplicar(8, 2)).thenReturn(16);

// 3. Instanciar la clase bajo prueba inyectando el mock
CalculadoraService svc = new CalculadoraService(calc);

// 4. Ejecutar el método que queremos probar
int result = svc.dobleDeLaSuma(3, 5);

// 5. Verificar el resultado
assertEquals(16, result);
```